Autonomous Network Exploitation via Partially Observable Markov Decission Processes

Reinforcement Learning Final Project

Iván Hernández Gómez

January 2026

Abstract

This report explores automated network exploitation with a focus on deep reinforcement learning techniques. It presents a conceptual overview of Microsoft's CyberBattleSim framework and details experiments implementing a Deep Recurrent Q-Learning (DRQN) agent capable of autonomously exploiting network vulnerabilities. The study compares the performance of the DRQN agent with a standard Deep Q-Network (DQN), highlighting the differences between methods designed for partially observable and fully observable environments. The results show that while the DRQN agent performs well on the training environment, it struggles to generalize to unseen networks, likely due to limitations in state representation, the simplicity of the training setup, and the lack of native handling of the network's graph structure. These findings provide insights into the challenges of applying reinforcement learning to real-world network exploitation and suggest avenues for improving agent generalization and environment modeling.

1 Introduction

According to the National Institute of Standards and Technology (NIST), computer network exploitation (NCE) refers to the enabling operations and intelligence collection capabilities conducted through the use of computer networks to gather data from target or adversary information systems or networks. In other words, it encompasses activities aimed at gaining access to computer systems in order to collect and leverage sensitive information. For instance, let us consider a simple ARP spoofing attack, in which an attacker attempts to trick users on a network into believing that it is the network access point (see Figure 1). The attacker's goal is to act as a Man-in-the-Middle (MitM), thereby receiving and analyzing all traffic exchanged by the users. In NCE terms, this attack (1) exploits the network ARP vulnerability to (2) collect sensitive information, which in this case consists of user traffic. Another example is a successful phishing attack targeting an employee of a company, which can grant an attacker an

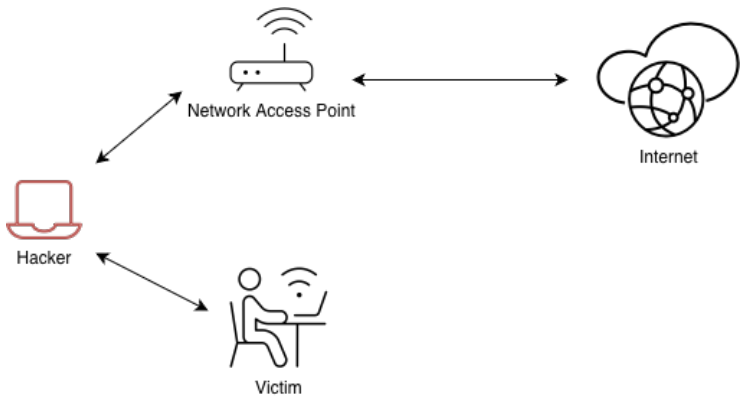

Figure 1: ARP spoofing attack

entry point into the private corporate network. From there, the attacker may move laterally, exploit local vulnerabilities, and employ other CNE techniques.

The number of different ways in which one can gain unauthorized access to a computer network is difficult to quantify and depends on the specific characteristics of the network. Consequently, companies and agencies often conduct adversary simulations (or red team exercises), carried out by experts, in which they simulate CNE techniques on their own networks in order to identify vulnerabilities. The aim of this project is to explore how gamifying this process into a Reinforcement Learning (RL) task can provide valuable and insightful results, all based on the Microsoft-proposed framework CyberBattleSim (Microsoft Defender Research Team, 2021).

2 Methodology

2.1 Network Exploitation as a RL task

The network exploitation task can be modeled as a sequential decision-making problem and addressed using reinforcement learning methods. An agent must take a sequence of actions, such as exploiting a local vulnerability on a computer through the execution of malware, in order to gain privileges, access other nodes, or steal relevant information. These achievements can be modeled using rewards. In a realistic scenario, where the agent is trained against a real network, the agent does not have full knowledge of the environment, nor does it have access to a generative model. However, it can interact with the network through actions (generate samples from complete trajectories) and forget all the gathered information and start over (reset the simulation when desired). For this reason, the task can be characterized as a resettable episodic interaction (see Figure 2).

Instead of developing a framework capable of autonomously exploiting emu-

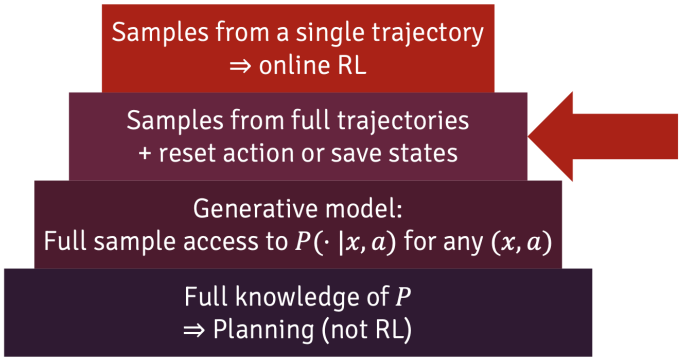

Figure 2: Levels of sample access in reinforcement learning. Source: G. Neu, *Reinforcement Learning*, lecture slides, Universitat Pompeu Fabra, 2025.

lated networks, Microsoft opted for a simulated environment, which tends to be lightweight, fast, abstract, and more controllable, making it more amenable to experimentation with reinforcement learning. In this way, they introduced *CyberBattleSim*, an OpenAI Gym–based framework for developing agents capable of autonomously exploiting and leveraging vulnerabilities in a network.

2.1.1 CyberBattleSim: Environment

The environment consists of a directed graph, where each node represents a computer in the enterprise network. An edge represents an open connection between two nodes, with a label identifying the connection, e.g., SSH, HTTP, etc. Each node stores several pieces of information that allow a rich representation of the network. Some of the attributes stored include:

- List of port/protocol the node is listening to
- List of known vulnerabilities for the node
- Intrinsic value of the node (translates into a reward if the node gets owned)
- Properties of the nodes, some of which can imply further vulnerabilities
- Firewall configuration of the node
- Flag indicating if the node is owned by the attacker
- Escalation level

The attributes not stated in this list refer to further capabilities of the framework not used in this project, specifically to presence of an active cyberdefense agent. As an illustrative example, we examine the properties of a simple network graph composed of two computer nodes (see Figure 3). One of the nodes is

infected and controlled by the agent (attacker), while the other remains uncompromised. As shown in the figure, the agent can exploit a local vulnerability on the compromised node to obtain leaked credentials, which then allow the establishment of a legitimate RDP connection to the second node. The reward for capturing the second node is 51.

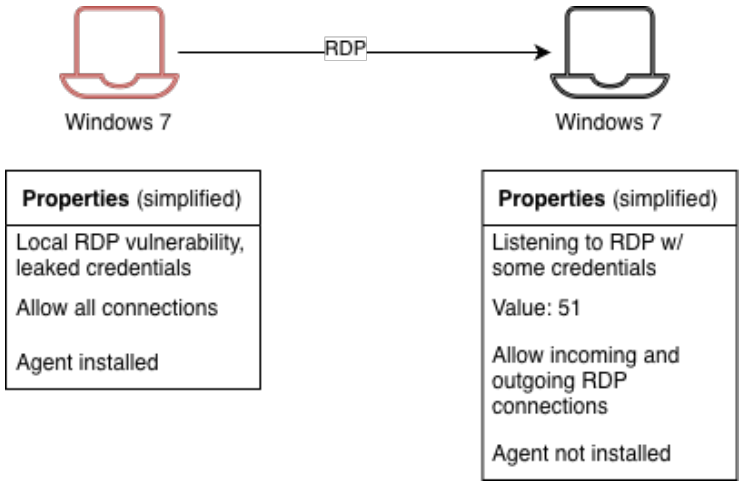

Figure 3: Properties of an infected (in red) and of a not-infected node

The framework is based on a post-breach assumption; that is, a node is always initially owned by the agent. This is a plausible and necessary precondition for taking actions within the network. In Figure 4, we can observe an example of a full CyberBattleSim network, with the owned node highlighted in red.

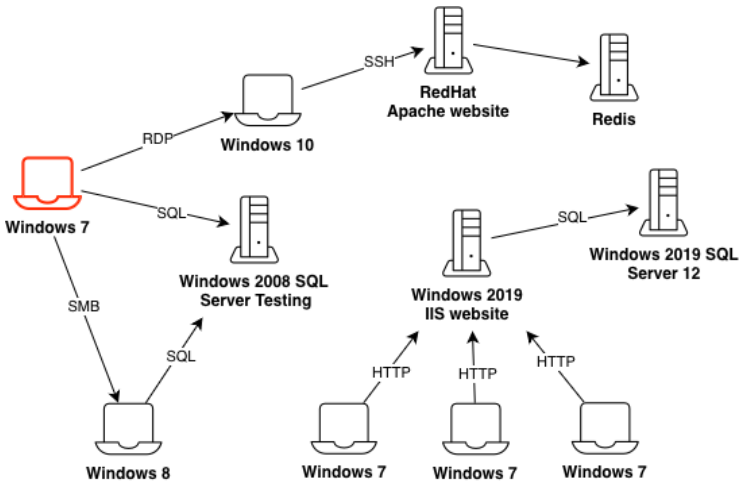

Figure 4: Full CyberBattleSim network

2.1.2 CyberBattleSim: Observations

In a realistic scenario the agent does not have full knowledge of the state of the network, but just recieves partial observation. This observation ecompases information from the outcome of the action just executed, and the state information aggregated over all actions executed so far. The overall structure of an observation is the following:

Immediate outcome

- Number of discovered nodes
- Lateral move flag
- Found customer data flag
- Probe result
- Escalation level
- Leaked credentials
- Valid actions

Aggregated information

- Discovered credentials
- Discovered nodes count
- Discovered nodes properties
- Discovered nodes privileges

As we will see later, not all information is taken into account when selecting an action, which is a topic for further research. The selected features are concatenated and converted into tensors to produce the state representation that our deep learning models will process. In Figure 5, we can observe the agent's initial knowledge (left), which is nonexistent, as well as the agent's network knowledge after some steps (right)

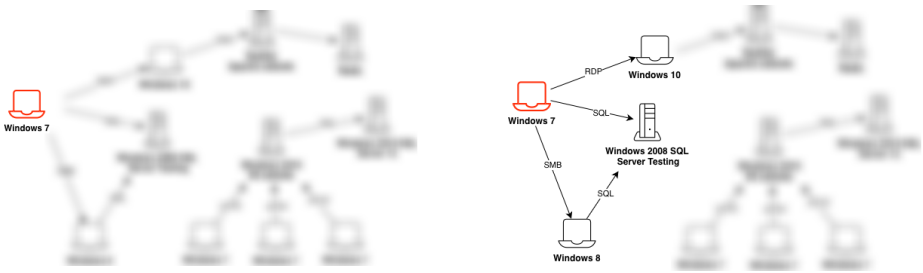

Figure 5: Partial observability of the network

2.1.3 CyberBattleSim: Actions

There are three primary types of actions an agent can take: exploiting a local vulnerability, exploiting a remote vulnerability, and connecting to a remote machine. Exploiting a local vulnerability requires a node and a known vulnerability. Exploiting a remote vulnerability requires a vulnerability, a source node, and a target node. Connecting to a remote machine requires a source node, a

target node, and valid credentials to establish the connection. Vulnerabilities are represented as discrete IDs, and credentials (user-password pairs) are unified into a single string.

2.1.4 CyberBattleSim: Rewards

The simulation includes positive rewards for successful actions and penalties for invalid actions, which generate suspicious behavior that could be detected by a hypothetical defender—for example, probing a non-listening port. Some examples of rewarded and penalized actions are:

Rewarded	**Penalized**
• Successfull attack	• Scanning an unopen port
• Node discovered	• Failed remote exploit
• Credential discovered	• Failed local exploit
• Property discovered	• Wrong password
• ...	• ...

Additionally, as stated previously, each node has an associated value that is awarded to the agent upon taking ownership. This allows the agent to be rewarded for exploiting highly valuable nodes, such as an SQL server, while receiving less reward for exploiting low-value nodes, such as a test machine.

2.2 Deep Q-Learning

Deep Q-Learning extends classical Q-learning by introducing a neural network to approximate the action–value function. Instead of maintaining a tabular representation, a Deep Q-Network (DQN) learns a parametric approximation $Q_\theta(s, a)$ that maps state–action pairs to their expected cumulative reward. The CyberBattleSim framework provides a reference implementation of a Deep Q-Network, which is adopted in this work as a baseline and used to compare against our Deep Recurrent Q-Network (DRQN) implementation. This comparison is conducted at the model level, with each model operating under its respective formulation (MDP for DQN and POMDP for DRQN).

To ensure training stability, Deep Q-Learning relies on several key techniques, that we have used as well for developing the Deep Recurrent Q-Network:

1. **Experience Replay.** At each time step, the agent stores its transition $e_t = (s_t, a_t, r_t, s_{t+1})$ in a replay memory D. During training, minibatches of transitions are sampled uniformly from D. This mechanism breaks temporal correlations between consecutive samples and improves data efficiency.

2. **Target Network.** A separate target network is used to compute the target Q-values during training. This network is identical in architecture

to the main network but is parameterized by θ', which is held fixed for a number of iterations. Periodically (every 10,000 iterations in our setup), the target network parameters are updated to match the main network parameters θ. This decoupling stabilizes learning by preventing the network from chasing its own moving targets.

3. **Adaptive Learning Rate.** An adaptive optimization method such as RMSProp or ADADELTA is employed to maintain a per-parameter learning rate α. These methods adjust α based on the history of gradient updates.

The provided implementation uses the following features concatenated as input to the Deep Q Network:

- Number of discovered nodes that are not owned
- Number of successfull actions at the target node
- Number of failed actions at the target node
- Target node properties
- Target node age

2.3 Deep Recurrent Q-Learning

Deep Recurrent Q-Learning extends Deep Q-Learning by replacing the feedforward Q-network with a recurrent neural network, typically based on architectures such as LSTMs or GRUs. The recurrent structure enables the network to approximate the action–value function by conditioning not only on the current observation but also on a history of past observations and actions, encoded in a latent hidden state.

As a result, Deep Recurrent Q-Networks (DRQNs) are well suited for environments characterized by partial observability. By maintaining an internal hidden representation that summarizes past interactions, the recurrent network implicitly estimates a belief over the underlying environment state. In this sense, DRQNs can be interpreted as operating within a Partially Observable Markov Decision Process (POMDP) formulation, where the hidden state serves as a learned approximation of the belief state in classical POMDP theory.

From a theoretical perspective, solving POMDPs exactly is known to be computationally intractable: finite-horizon POMDP planning is PSPACE-complete, while the infinite-horizon case is undecidable (Kaelbling, Littman, & Cassandra, 1998). Deep Recurrent Q-Learning does not attempt to solve the POMDP optimally in the classical sense; instead, it provides a practical, scalable approximation that leverages recurrent function approximation to address partial observability in complex, high-dimensional environments.

The implemented Deep Recurrent Q-Learning (DRQL) architecture is based on the methodologies described in (Hausknecht & Stone, 2015). Several ideas from that work are key to understanding the development of the agent.

First, the use of recurrence does not confer a systematic advantage during training, but the recurrent network is better able to adapt at evaluation time if the quality of observations changes. To isolate the effects of recurrency, we minimally modify the standard Deep Q-Network (DQN) architecture, replacing only the first fully connected layer with a recurrent LSTM layer of the same size.

Training a recurrent, convolutional network requires each backward pass to include many time-steps of game observations and corresponding target values. Additionally, the LSTM's initial hidden state may either be zeroed at the start of each update or carried forward from its previous values. In this implementation, we adopt the Bootstrapped Random Updates strategy: episodes are selected randomly from the replay memory, and updates begin at random points within the episode, proceeding for only a limited number of unroll iterations (i.e., one backward pass). At each timestep, target values are generated from the target Q-network $\hat{Q}$, and the RNN's initial hidden state is zeroed at the start of the update.

3 Results

The agents were trained on 4 episodes of 500 steps each and evaluated on 3 episodes of the same length. During training, it was observed that the random bootstrapping mechanism used in DRQL significantly slows down learning. This is because each update requires processing a new sequence of transitions, unlike standard DQL where a single transition suffices for each update. The training comparison is shown in Figure 6.

Increasing the number of training steps for DQL from 500 to 1500 per episode dramatically improves its performance, bringing it closer to that of DRQL. Despite this improvement, DRQL still requires more time to train due to the additional computational overhead of processing sequences of transitions. This comparison is illustrated in Figure 7.

One of the main objectives of training a reinforcement learning agent is to enable it to generalize to unseen environments or scenarios, making effective decisions based on the (possibly partial) observed state. To evaluate this capability, we compare the performance of the trained DRQL agent against a baseline heuristic agent over a single episode (see Figure **??**). The baseline agent executes random actions and exploits credentials whenever available, providing a simple yet informative benchmark for assessing whether the trained agent can outperform a basic heuristic strategy.

The results indicate that the DRQL agent was not able to achieve significant improvements and was outperformed by the heuristic agent. Several factors may contribute to this outcome, which will be analyzed in detail in the Conclusions section.

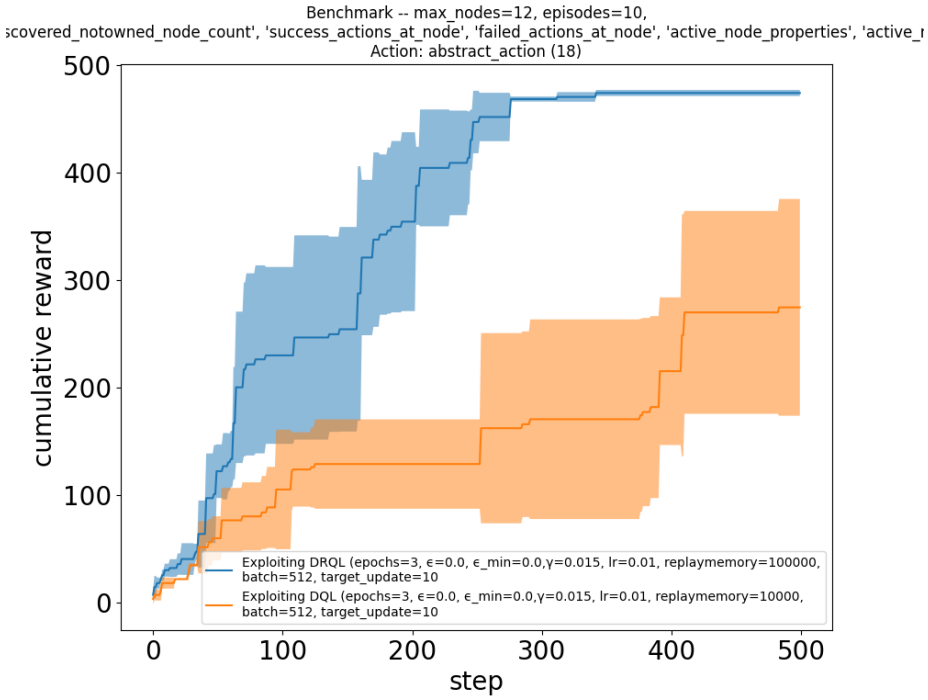

Figure 6: Performance of DQL and DRQL on the training environment when trained with the same number of steps.

4 Conclusions

The results of this study provide several insights into the application of Deep Recurrent Q-Learning (DRQL) to network exploitation scenarios. On the training environment, the agent achieved good performance, demonstrating that it is capable of learning from sequential interactions when the environment is well-known. However, when evaluated on an unseen environment, the agent's performance was poor. Several factors may explain this outcome. First, the state representation may have been insufficient or noisy, limiting the agent's ability to determine optimal actions. Second, the approach did not natively leverage the underlying graph structure of the network, instead relying on simple aggregation of data. Finally, the training network was significantly simpler than the evaluation environment, suggesting that a more representative or complex training setup could improve generalization to unseen networks.

This project also offered important practical and conceptual learnings. By applying knowledge from the lectures to a real problem, such as network exploitation, we gained first-hand experience in designing reinforcement learning agents for applied tasks. The gamification of the problem provided an intuitive

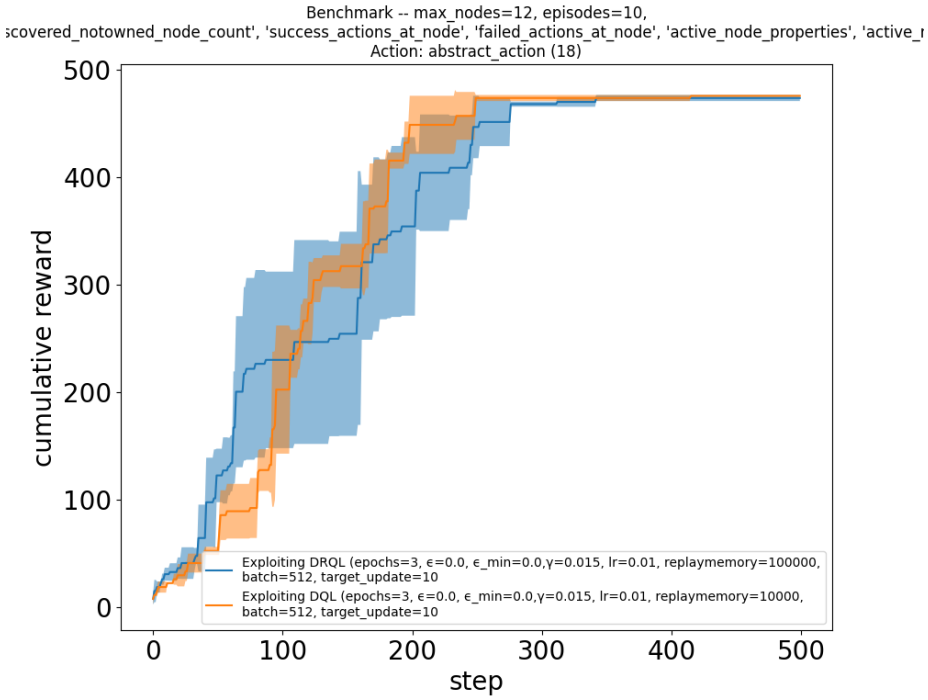

Figure 7: Performance of DQL and DRQL on the training environment when trained with different numbers of steps per episode.

introduction to creating RL environments for real-world applications. Additionally, we deepened our understanding of stability techniques in Deep Q-Network training, explored the OpenAI Gym interface, and performed a detailed analysis of the CyberBattleSim framework, learning best practices from its design. Finally, a literature review of Partially Observable Markov Decision Processes (POMDPs) provided a conceptual foundation for understanding alternative approaches to handling partial observability in reinforcement learning.

Overall, the project highlighted both the potential and the limitations of DRQL in complex network environments, pointing to several avenues for future improvements in state representation, environment modeling, and training strategies.

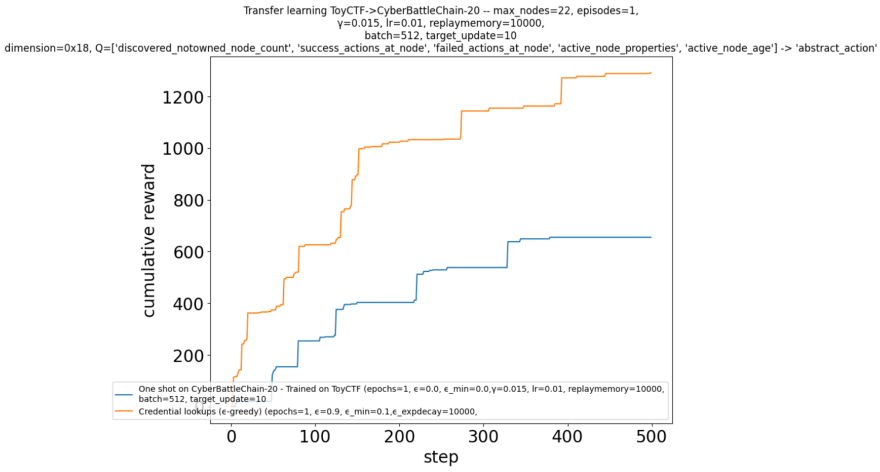

Figure 8: Comparison of the trained DRQL agent with a heuristic baseline agent on an unseen episode.

References

Hausknecht, M., & Stone, P. (2015). Deep recurrent q-learning for partially observable MDPs. In *Proceedings of the aaai conference on artificial intelligence* (Vol. 29). Austin, Texas, USA.

Kaelbling, L. P., Littman, M. L., & Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, *101*(1-2), 99–134. doi: 10.1016/S0004-3702(98)00023-X

Microsoft Defender Research Team. (2021). *Cyberbattlesim.* GitHub repository. Retrieved from `https://github.com/microsoft/cyberbattlesim` (Created by Christian Seifert, Michael Betser, William Blum, James Bono, Kate Farris, Emily Goren, Justin Grana, Kristian Holsheimer, Brandon Marken, Joshua Neil, Nicole Nichols, Jugal Parikh, Haoran Wei)